AI1	Dokumentacja projektu
Autor	Ewelina Kaniewska, 125129
Kierunek, rok	Informatyka, II rok, st. stacjonarne (3,5-l)
Temat projektu	*Sprzedaż słodyczy na wagę*

Spis treści

1. **Wstęp**
2. **Narzędzia i technologie**
3. **Baza danych**
4. **Graficzny interfejs użytkownika**
5. **Uruchomienie aplikacji**
6. **Funkcjonalności aplikacji**

1. Wstęp

Projektowana aplikacja internetowa ma na celu zarządzanie sprzedażą słodyczy na wagę. Jest to system przeznaczony zarówno dla właścicieli sklepów ze słodyczami, jak i dla ich klientów. Aplikacja umożliwia sprawne zarządzanie asortymentem i dostawcami. Dzięki aplikacji możliwe jest przeglądanie słodyczy i dostawców dla zwykłych użytkowników oraz użytkowników niezalogowanych. Administrator posiada prawo do dodawania, usuwania, aktualizowania i przeglądania wszystkich zasobów.

2. Narzędzia i technologie

Języki programowania

- **PHP:** Serwerowy język skryptowy używany do tworzenia logiki aplikacji webowej
  - Wersja: PHP 8.2.12
  - Licencja: PHP License (Open Source)
  - Dokumentacja
- **HTML:** Standardowy język znaczników używany do tworzenia struktury stron internetowych.
  - Wersja: HTML5
  - Licencja: Open (no specific license)
  - Dokumentacja
- **JavaScript:** Język skryptowy używany do tworzenia interaktywnych efektów na stronach internetowych.
  - Dokumentacja

Frameworki i biblioteki

- **Laravel:** Framework PHP używany do tworzenia aplikacji webowych.
  - Wersja: Laravel 11.7.0
  - Licencja: MIT License (Open Source)
  - Dokumentacja
- **Bootstrap:** Biblioteka CSS używana do tworzenia responsywnych i stylowych interfejsów użytkownika.
  - Wersja: Bootstrap 5
  - Licencja: MIT License (Open Source)
  - Dokumentacja

Bazy danych

- **MariaDB:** System zarządzania relacyjnymi bazami danych używany do przechowywania danych aplikacji.
  - Wersja: 10.4.32-MariaDB
  - Licencja: GPL License (Open Source)
  - Dokumentacja

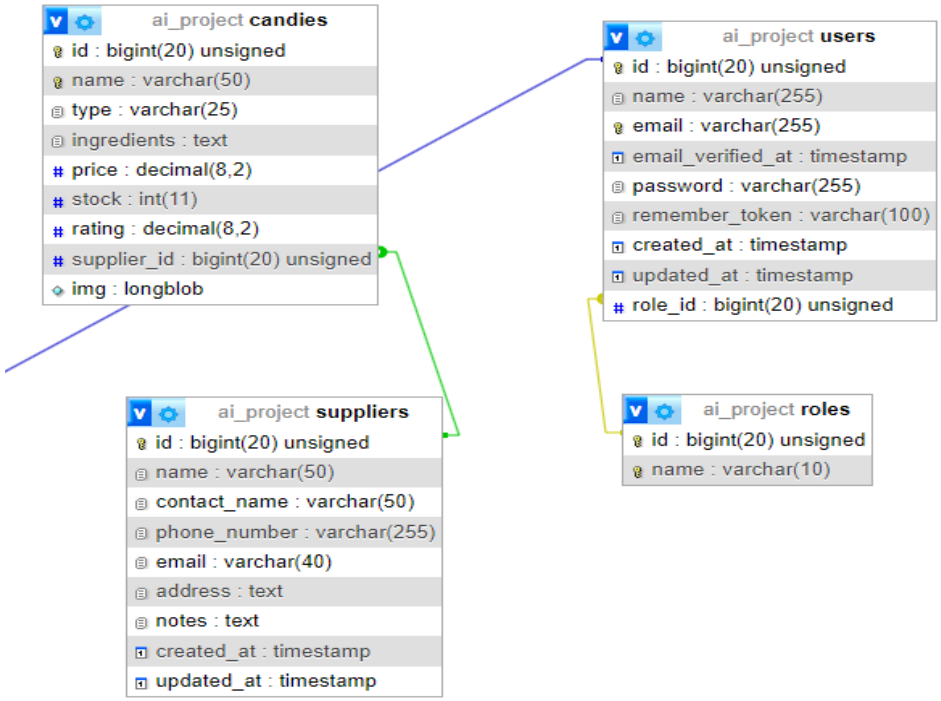
Środowiska programistyczne

- **XAMPP:** Pakiet oprogramowania zawierający Apache, MariaDB, PHP używany do lokalnego uruchamiania serwera webowego.
  - Wersja: 8.2.12
  - Licencja: GPL License (Open Source)
  - Dokumentacja
- **Visual Studio Code:** Edytor kodu źródłowego używany do programowania aplikacji.
  - Wersja: 1.89.1
  - Licencja: MIT License (Open Source)
  - Dokumentacja

Narzędzia pomocnicze

- **Composer:** Menedżer zależności dla PHP używany do zarządzania bibliotekami i pakietami.
  - Wersja 2.7.6
  - Licencja: MIT License (Open Source)
  - Dokumentacja
- **Git:** System kontroli wersji używany do zarządzania kodem źródłowym.
  - Wersja 2.45.1
  - Licencja: GPL License (Open Source)
  - Dokumentacja

3. Baza danych



Rysunek 1 Schemat ERD Bazy Danych

Baza danych aplikacji zarządzającej słodyczami na wagę składa się z czterech głównych tabel: candies (słodycze), suppliers (dostawcy), users (użytkownicy) oraz roles (role).

Tabela candies

Tabela **candies** przechowuje informacje o dostępnych słodyczach. Każdy rekord w tabeli reprezentuje jeden rodzaj słodyczy.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator słodyczy.
- **name** (string, 50, unique): Nazwa słodyczy.
- **type** (string, 25): Rodzaj słodyczy.
- **ingredients** (text): Składniki słodyczy.
- **price** (decimal): Cena za kilogram.

- **stock** (int): Ilość dostępna w magazynie (w kilogramach).
- **rating** (decimal): Ocena słodyczy.
- **supplier_id** (int, foreign key): Identyfikator dostawcy, który dostarcza słodycze.
- **img** (LONGBLOB): Zdjęcie słodyczy w formacie binarnym.

id	name	type	ingredients	price	stock	rating	supplier_id	img
1	Truskawkowe landrynki	Landrynki	Cukier, syrop glukozowy, aromat truskawkowy, barwn...	23.99	200	4.20	1	[BLOB - 11.8 KB]

Rysunek 2 Przykładowe dane w tabeli candies

Tabela suppliers

Tabela **suppliers** przechowuje informacje o dostawcach słodyczy. Każdy rekord w tabeli reprezentuje jednego dostawcę.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator dostawcy.
- **name** (string, 50): Nazwa dostawcy.
- **contact_name** (string, 50): Imię i nazwisko osoby kontaktowej.
- **phone_number** (string): Numer telefonu kontaktowego.
- **email** (string, 40): Adres e-mail dostawcy.
- **address** (text): Adres dostawcy.
- **notes** (text): Dodatkowe notatki.

id	name	contact_name	phone_number	email	address	notes	created_at	updated_at
1	Runte-Bergstrom	Miss Letha Bartoletti	352-549-3316	hershel64@example.com	63204 Monica Course Hicklechester, VA 60072	Non ea unde aut voluptates reiciendis quisquam occ...	2024-01-22 01:13:40	2024-04-21 03:48:26

Rysunek 3 Przykładowe dane w tabeli suppliers

Tabela users

Tabela **users** przechowuje informacje o użytkownikach aplikacji. Każdy rekord w tabeli reprezentuje jednego użytkownika.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator użytkownika.
- **name** (string, 50): Nazwa użytkownika.
- **email** (string, 100, unique): Adres e-mail użytkownika.
- **password** (string, 255): Hasło użytkownika (przechowywane jako hash).
- **role_id** (int, foreign key): Identyfikator roli, którą posiada dany użytkownik (admin, user).

id	name	email	email_verified_at	password	remember_token	created_at	updated_at	role_id
1	Jan	jan@email.com	NULL	$2y$12$.WjCFfT0I9KZc7JTDJJt0OGbGO8x5qAP8Chan2IWgS4...	NULL	NULL	NULL	1

Rysunek 4 Przykładowe dane w tabeli users

Tabela roles

Tabela **roles** przechowuje informacje o rolach użytkowników w aplikacji. Każdy rekord w tabeli reprezentuje jedną rolę, która może być przypisana użytkownikowi. Tabela ta pozwala na łatwe zarządzanie uprawnieniami użytkowników.

Struktura tabeli:

- **id** (int, primary key, auto-increment): Unikalny identyfikator roli.
- **name** (string, 50, unique): Nazwa roli.

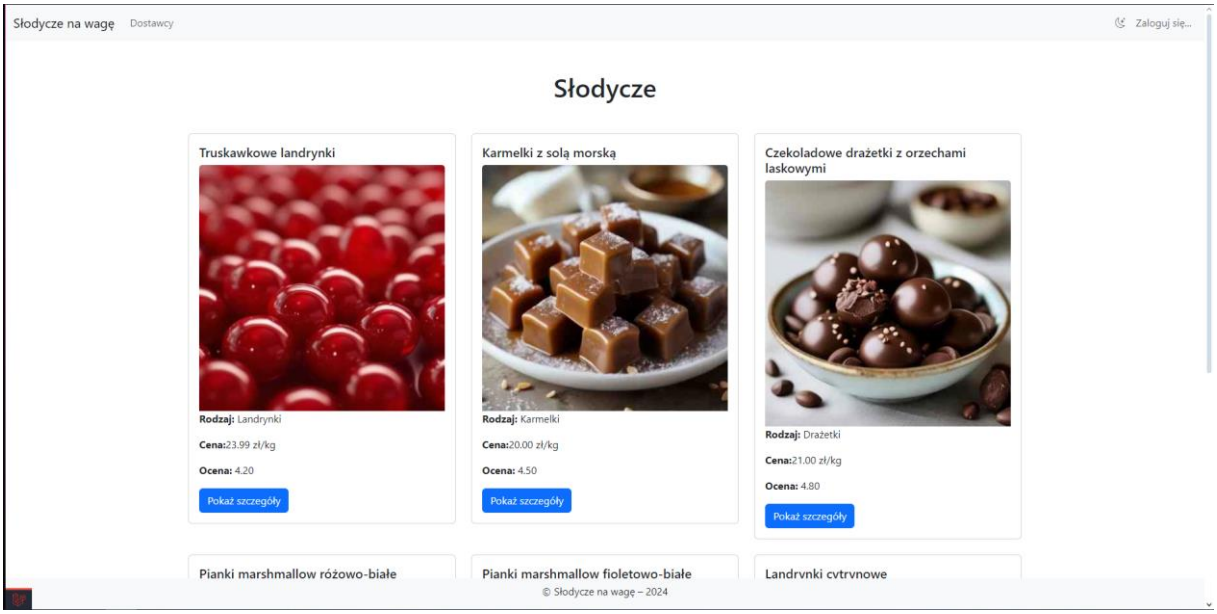
id	name
1	admin

Rysunek 5 Przykładowe dane w tabeli roles

Baza danych aplikacji zarządzającej słodyczami na wagę składa się z czterech głównych tabel: **candies**, **suppliers**, **users** i **roles**. Tabele te są ze sobą powiązane relacjami, co umożliwia efektywne zarządzanie informacjami o słodyczach, ich dostawcach oraz użytkownikach aplikacji. Tabela **candies** przechowuje szczegóły dotyczące każdego rodzaju słodyczy, tabela **suppliers** zawiera informacje o dostawcach, tabela **users** przechowuje dane o użytkownikach aplikacji, a tabela **roles** o ich rolach.

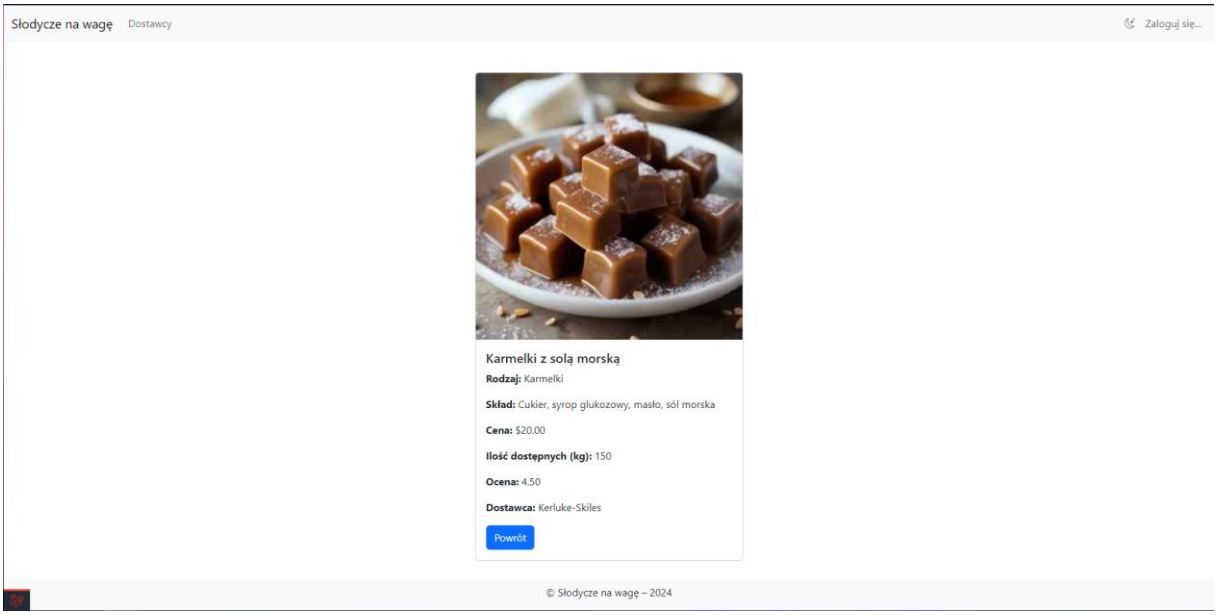
4. Graficzny interfejs użytkownika

Uruchamiając aplikację użytkownik widzi pierwszy - główny - z dostępnych widoków, który przedstawia dostępne słodycze w formie kart.



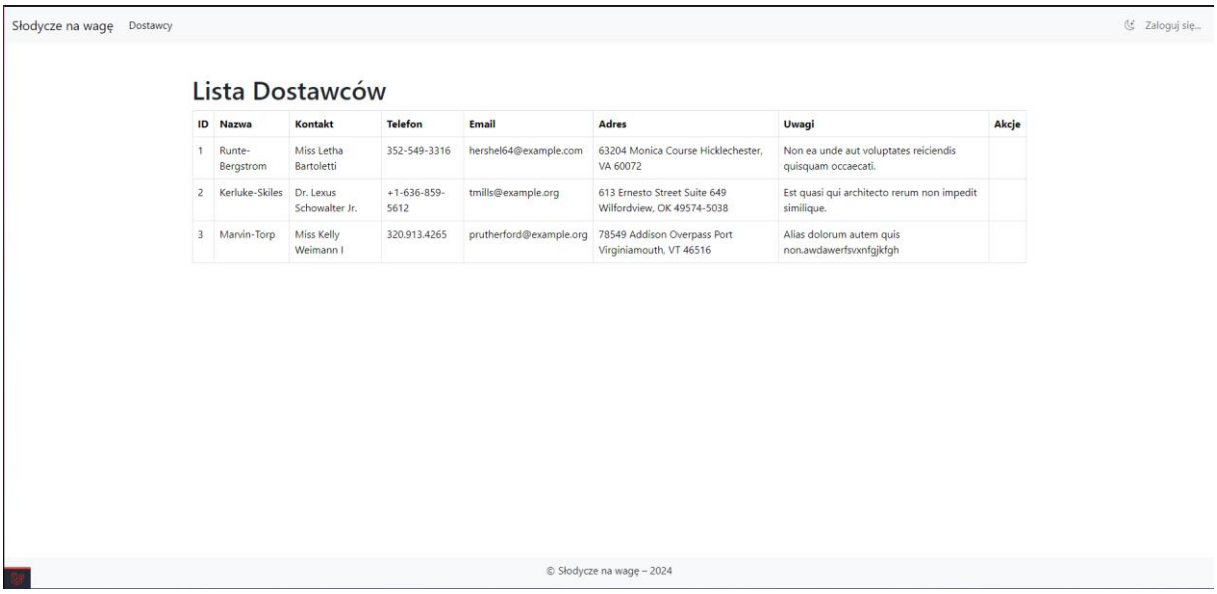
Rysunek 6 Główny widok aplikacji

Po kliknięciu w przycisk pokaż szczegóły użytkownik przechodzi do widoku pojedynczego produktu.



Rysunek 7 Widok pojedynczego produktu

Klikając w segment ‘Dostawcy’ w lewym górnym rogu strony użytkownik przechodzi do listy dostawców słodyczy



ID	Nazwa	Kontakt	Telefon	Email	Adres	Uwagi	Akcje
1	Runte-Bergstrom	Miss Letha Bartoletti	352-549-3316	hershel64@example.com	63204 Monica Course Hicklechester, VA 60072	Non ea unde aut voluptates reiciendis quisquam occaecati.	
2	Kerluke-Skiles	Dr. Lexus Schowalter Jr.	+1-636-859-5612	tmills@example.org	613 Ernesto Street Suite 649 Wilfordview, OK 49574-5038	Est quasi qui architecto rerum non impedit similique.	
3	Marvin-Torp	Miss Kelly Weimann I	320.913.4265	prutherford@example.org	78549 Addison Overpass Port Virginiamouth, VT 46516	Alias dolorum autem quis non.awdawerfsvxnfgjkfgh	

Rysunek 8 Lista dostawców

5. Uruchomienie aplikacji

Do uruchomienia aplikacji potrzebne jest zainstalowanie odpowiedniego oprogramowania

- XAMPP
- Composer

Instalację programu xampp należy przeprowadzić krok po kroku za pomocą instalatora (na dysku systemowym).

Następnie należy przygotować bazę danych. Aby to wykonać trzeba uruchomić xamppa i włączyć MySQL i Apache. Następnie z poziomu panelu administracyjnego lub z poziomu konsoli utworzyć bazę danych. Komenda do utworzenia bazy z poziomu konsoli wygląda następująco:
C:\xampp\mysql\bin\mysql -uroot -e "CREATE DATABASE IF NOT EXISTS AI_PROJECT;" pod warunkiem, że xampp został zainstalowany na dysku systemowym (C).

Aby zainstalować composer należy z terminala wykonać komendę *composer install* w katalogu projektu.

Po pomyślnej instalacji potrzebnego oprogramowania należy wykonać następujące komendy z konsoli będąc w katalogu projektu:

php artisan migrate:fresh --seed
php artisan key:generate
php artisan storage:link

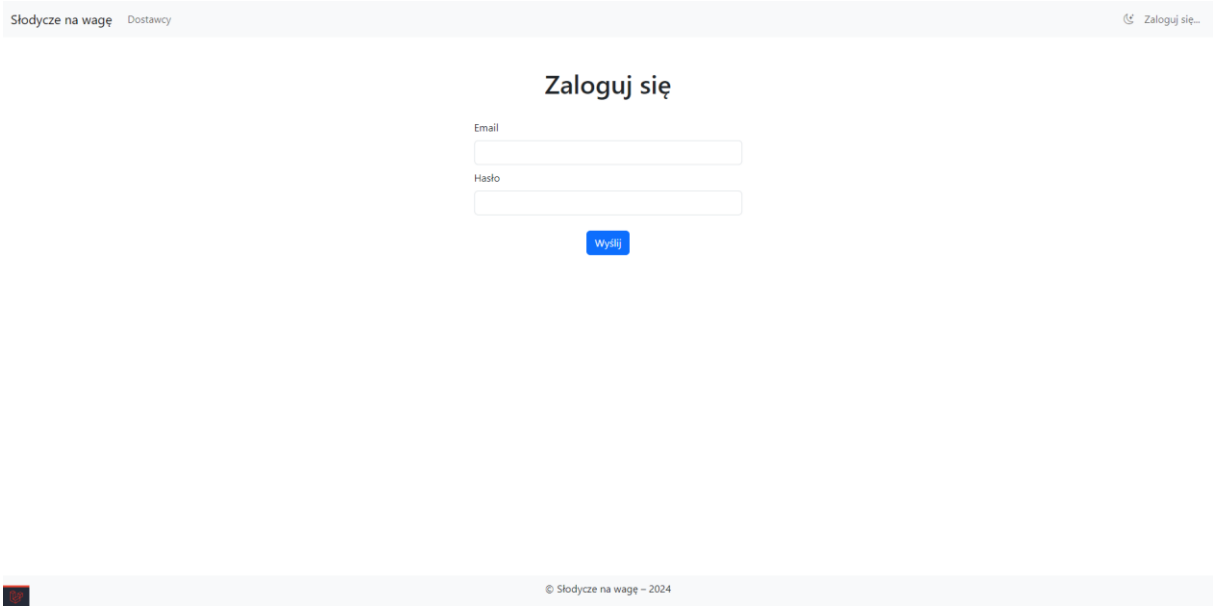
Teraz baza danych jest gotowa do korzystania z aplikacji.
Należy jeszcze tylko uruchomić serwer komendą
php artisan serve
w katalogu aplikacji.

Aby dostać się do głównego widoku projektu należy w pasku adresu przeglądarki wpisać *http://127.0.0.1:8000*.

6. Funkcjonalności projektu

- Logowanie

Do formularza logowania prowadzi odnośnik w prawym górnym rogu.



Rysunek 9 Formularz logowania

Dane aministratora:

Email: jan@email.com

Hasło: 1234

Logowanie jest obsługiwane przez kontroler AuthController. Kontroler ten zawiera metody odpowiadające za wyświetlanie formularza logowania, obsługę procesu logowania, a także wylogowywanie użytkowników.

```
public function login()
{
    if (Auth::check()) {
        return redirect()->route('candies.index');
    }
    return view('auth.login');
}
```

Rysunek 10 Metoda login

Cel: Wyświetlenie formularza logowania.

Działanie: Metoda sprawdza, czy użytkownik jest już zalogowany za pomocą Auth::check().

Jeśli użytkownik jest zalogowany, następuje przekierowanie do strony głównej aplikacji (candies.index).

Jeśli użytkownik nie jest zalogowany, wyświetlany jest widok formularza logowania (auth.login).

```
public function authenticate(Request $request)
{
    $credentials = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (Auth::attempt($credentials)) {
        $request->session()->regenerate();
        return redirect()->route('candies.index');
    }

    return back()->withErrors([
        'email' => 'The provided credentials do not match our records.',
    ])->onlyInput('email');
}
```

Rysunek 11 Metoda authenticate

Cel: Obsługa próby logowania.

Działanie:

Metoda najpierw waliduje dane wejściowe, sprawdzając, czy pola email i password są obecne i poprawne.

Następnie używa Auth::attempt($credentials), aby spróbować zalogować użytkownika za pomocą podanych danych.

Jeśli dane logowania są poprawne sesja użytkownika jest regenerowana.

Następuje przekierowanie do strony głównej aplikacji (candies.index).

Jeśli dane logowania są niepoprawne użytkownik jest przekierowany z powrotem do formularza logowania z komunikatem o błędzie.

```php
public function logout(Request $request)
{
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();
    return redirect()->route('candies.index');
}
```

Rysunek 12 Metoda logout

Cel: Obsługa procesu wylogowywania użytkownika.

Działanie:

Metoda wylogowuje użytkownika za pomocą Auth::logout().

Następnie sesja użytkownika jest unieważniana ($request->session()->invalidate()) oraz generowany jest nowy token CSRF ($request->session()->regenerateToken()), co dodatkowo zabezpiecza aplikację.

Na koniec następuje przekierowanie do strony głównej aplikacji (candies.index).

- CRUD
  I. **CREATE**

     Tworzenie nowego produktu jest realizowane przez metody create i store kontrolera CandyConroller. Tylko administrator może dodawać nowe zasoby.

```blade
@can('is-admin')
        <a href="{{ route('candies.edit', $candy->id) }}" class="btn btn-warning">Edytuj</a>
        <form action="{{ route('candies.destroy', $candy->id) }}" method="POST" class="d-inline">
            @csrf
            @method('DELETE')
            <button type="submit" class="btn btn-danger">Usuń</button>
        </form>
    @endcan
```

Rysunek 13 Ukrycie przycisków dla zwykłych użytkowników

Ponadto zostało zastosowane zabezpieczenie, dzięki któremu tylko administrator może wykonać akcję dodawania produktu.

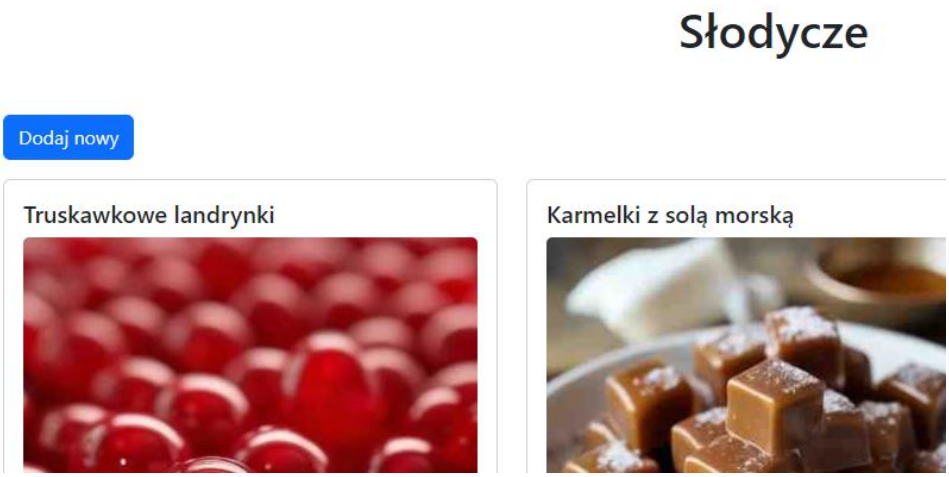
```
class StoreCandyRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     */
    public function authorize(): bool
    {
        return Gate::allows('is-admin');
    }
```

Rysunek 14 Zabezpieczenie przed dodaniem zasobu przez nieupoważnionego użytkownika

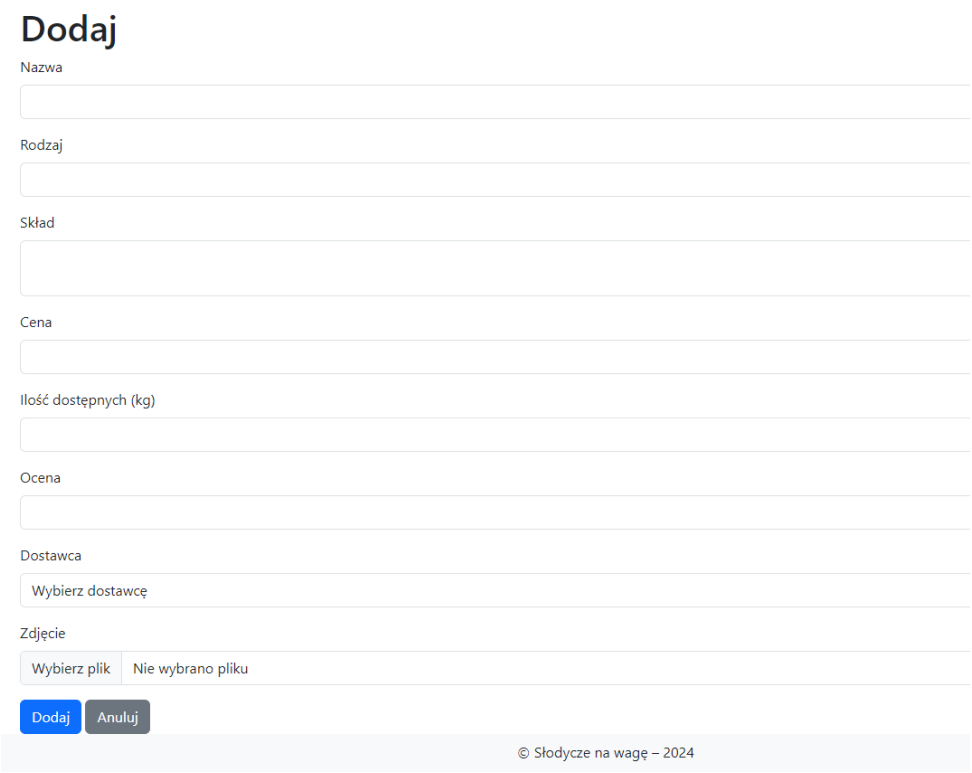
```
public function create()
{
    if (! Gate::allows('is-admin')) {
        abort(403);
    }

    $suppliers = Supplier::all();
    return view('candies.create', ['suppliers' => $suppliers]);
}
```

Rysunek 15 Zabezpieczenie przed wyświetleniem formularza dodawania



Rysunek 16 Administrator ma dostępny przycisk dodawania produktu

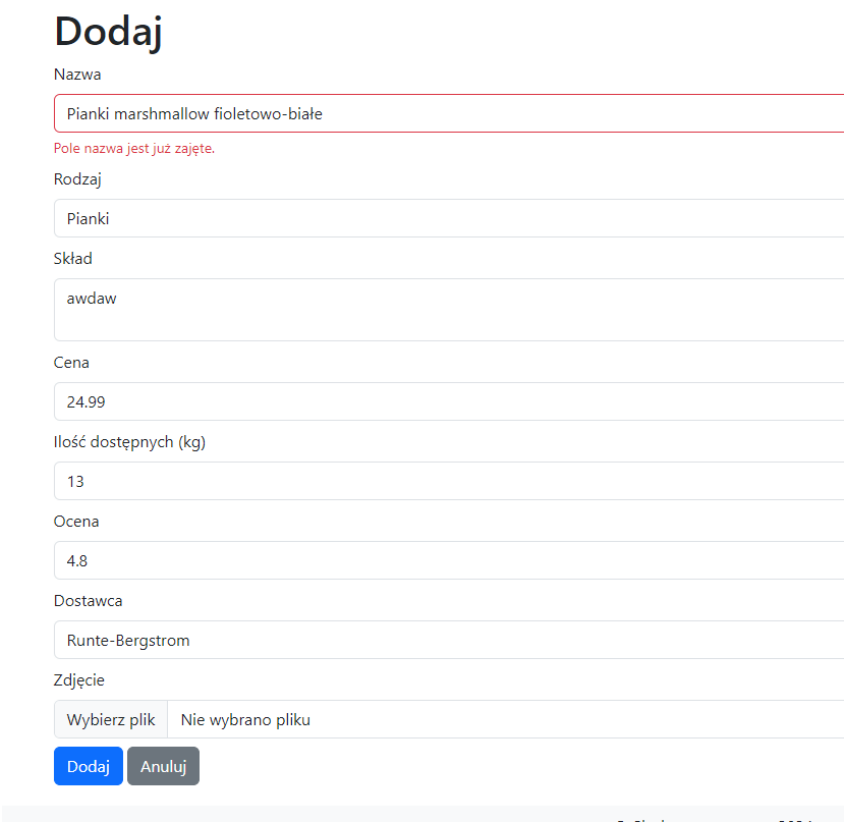


Rysunek 17 Formularz dodawania produktu

```
public function rules(): array
{
    return [
        'name' => 'required|unique:candies,name,|max:50',
        'type' => 'required|string|max:25',
        'ingredients' => 'required|string',
        'price' => 'required|numeric|between:0,9999.99',
        'stock' => 'required|integer|min:0',
        'rating' =>'required|numeric|between:0,5'
    ];
}
```

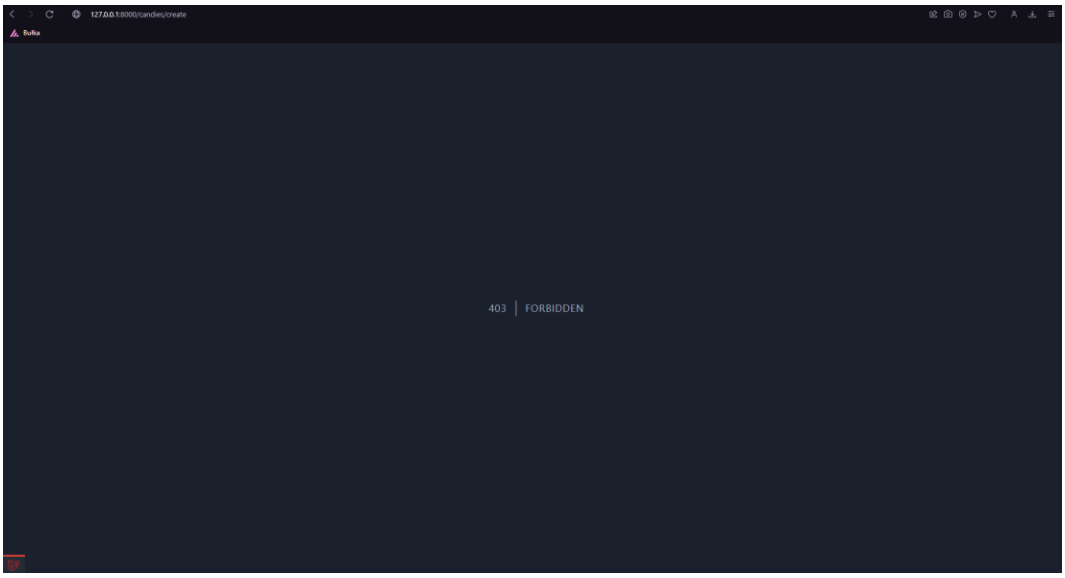
Rysunek 18 Walidacja danych

Dane są walidowane za pomocą metody rules() w StoreCandyRequest



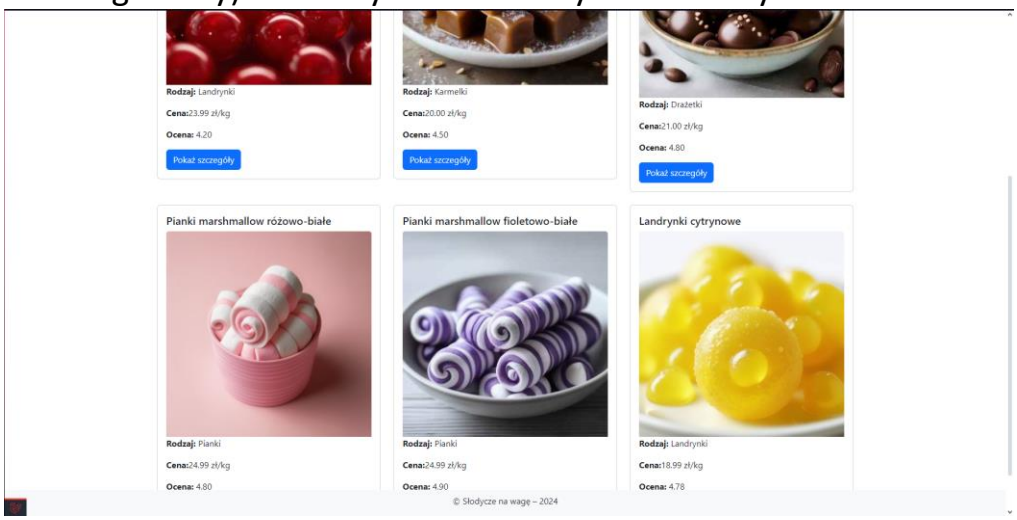
Rysunek 19 Przykład podania nazwy, która już istnieje

W przypadku, kiedy niezalogowany użytkownik (lub użytkownik, który nie jest adminem) spróbuje wejść na podstronę z formularzem dodawania produktu zostanie wyświetlony komunikat błędu autoryzacji.

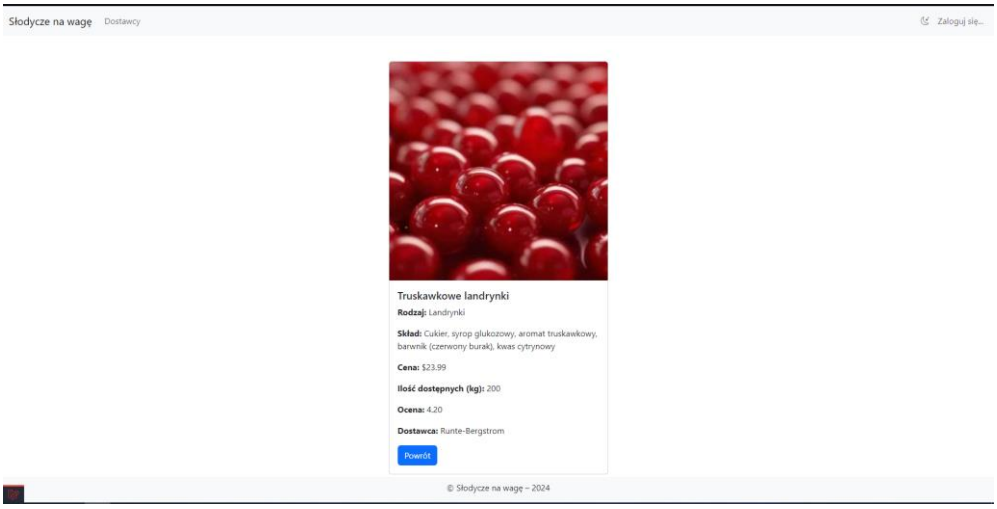


II. READ

Przeglądanie zasobów jest realizowane za pomocą metod index() i show($id) kontrolera CandyController. Każdy użytkownik (również niezalogowany) może wyświetlać wszystkie zasoby



Rysunek 20 Przeglądanie ogólnodostępnych zasobów



Rysunek 21 Wyświetlanie ogólnie dostępnego zasobu

III. UPDATE

Aktualizowanie danych jest realizowane za pomocą funkcji edit i update. Tylko administrator może dokonywać aktualizacji.

Truskawkowe landrynki

Rodzaj: Landrynki

Skład: Cukier, syrop glukozowy, aromat truskawkowy, barwnik (czerwony burak), kwas cytrynowy

Cena: $23.99

Ilość dostępnych (kg): 200

Ocena: 4.20

Dostawca: Runte-Bergstrom

Powrót Edytuj Usuń

© Słodycze na wagę – 2024

Rysunek 22 Przyciski dostępne dla administratora

Edytuj

Name

Truskawkowe landrynki

Type

Landrynki

Ingredients

Cukier, syrop glukozowy, aromat truskawkowy, barwnik (czerwony burak), kwas cytrynowy

Price

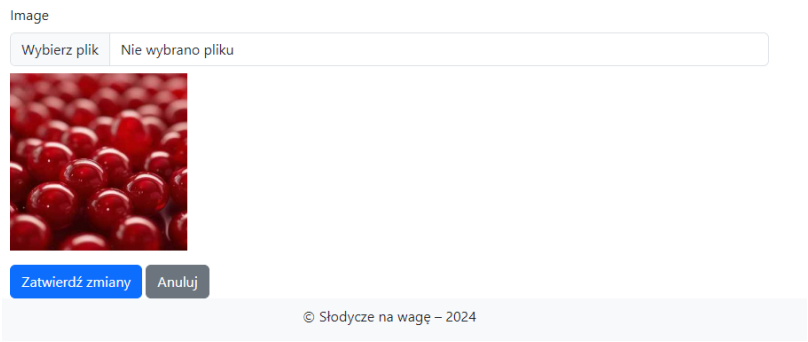
23,99

Stock

200

Rating

4,20



Rysunek 23 Formularz edycji

Zastosowano tutaj takie same mechanizmy zabezpieczające jak w przypadku dodawania zasobu: użytkownik, który nie jest adminem nie może zobaczyć przycisku ani wejść na podstronę z formularzem edycji.

IV. DELETE

Za usuwanie danych odpowiedzialna jest metoda delete kontrolera CandyController. Tylko admin może usunąć dane. Zabezpieczenia są takie same jak w przypadku Create i Update.

W taki sam sposób została przeprowadzona implementacja CRUD dla tabeli suppliers.